

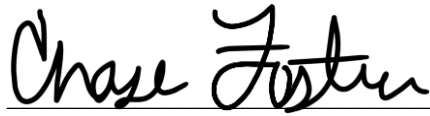
A Machine Learning Analysis of Factors Leading to Major League Baseball Postseason Berths

By

Chase Stephen Foster

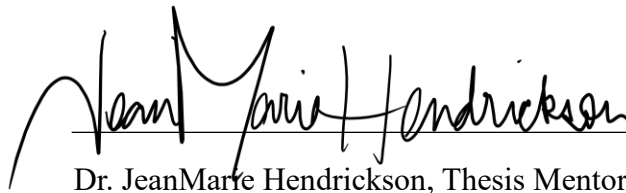
An Undergraduate Thesis Submitted in Partial Fulfillment
of the Requirements for the Global Citizens Scholars Honors Program

East Tennessee State University

 5/1/25

Chase S. Foster, Honors Student

Date

 5/2/25

Dr. JeanMarie Hendrickson, Thesis Mentor

Date

 5/3/25

Dr. Michael Garrett, Reader

Date

Abstract

Machine learning is a method that employs statistical algorithms to identify patterns and make predictions from data. This study applies machine learning techniques to analyze data from Major League Baseball (MLB) teams between 1998 and 2024, with the goal of determining which factors strongly influence a team's likelihood of reaching the postseason and in accurately predicting the teams that do and do not qualify for the postseason. Data exploration and unsupervised machine learning methods such as clustering were used to identify underlying patterns in team performance metrics and determine potential significant contributors to team success. Many different supervised learning methods were employed to develop predictive models. The dataset was randomly divided into a training set which was used to train the predictive models, and a test set which was used to evaluate the models' accuracy. Hierarchical and K-Means clustering were used to group similarly performing teams and identify variables that had great influence on the teams' performance. Logistic regression, KNN Classification, Linear Discriminant Analysis, Non-linear Functions, Decision Trees, Bagging, and Random Forest models were constructed to classify teams and evaluate the importance of the various predictors. Results indicate that ERA+, Runs Allowed, OPS+, and Runs Scored are the most significant contributors to postseason qualification. This stresses the importance of a balance between offensive and defensive strength and performance. A Random Forest model was selected and used to predict the outcomes of the 2025 season based on early-season data. By identifying key performance indicators, this research aims to offer insights into critical contributors to reaching the postseason in the MLB. This research also demonstrates the value of machine learning in sports analytics and highlights how different methods can be used to handle complex data to support strategic decision-making and forecasting in professional baseball.

Contents

1 Overview of Methods & Background Information	4
2 Introduction & Data Collection	14
3 Unsupervised Learning Methods	18
3.1 Principal Component Analysis	18
3.2 Clustering	19
3.3 Method Comparison	23
4 Supervised Learning Methods	24
4.1 Logistic Regression	24
4.2 KNN Classification	25
4.3 Linear Discriminant Analysis	26
4.4 Non-Linear Functions	27
4.5 Decision Trees, Random Forest, & Bagging	29
5 Model Evaluation & Selection	33
6 Prediction & Discussion	35
7 Conclusion	37
8 References	38

1 Overview of Methods & Background Information

Unsupervised learning is a type of machine learning in which the goal is to discover patterns, relationships, or groupings within the data. Unsupervised learning typically analyzes unlabeled data with no specified output. Thus, it is not used for directly generating models or for prediction. Rather, it is used to provide insights into the data which could then be used for further analysis in supervised machine learning. Supervised learning is a type of machine learning in which the goal is to generate models that accurately represent the data. In supervised learning, the data is labeled, and there are specified inputs and output. The data is also split randomly into a training set and a test set. The training set usually contains most of the data and is used to teach the model and allow it to learn patterns within the data. The test set contains much less of the original data and is used to evaluate the performance and accuracy of the model. The test set is unseen by the model; thus, it is used to determine how well the model generalizes. The model uses the inputs from the test set and predicts the outputs. The predicted outputs are then compared to the actual known outputs in the test set to grade how well the model performed. If the model performs well, it can be used for predictions on future data.

Principal Component Analysis (PCA) is an unsupervised learning method. It refers to a process for using principal components that capture the source of variability in the original data. Principal components are variables used to combat highly correlated variables in the data and encapsulate the most significant ones. Principal Component Analysis can also be used for data visualization and creating variables for supervised learning contexts. Principal Component Analysis determines a low-dimensional representation of the data while containing much of the variation. PCA finds a small number of dimensions that maintain as much of the variation in the

data set as possible. Each is a linear combination of the variables in the data set. To calculate principal components, the variables must be centered to have a mean of 0. For this dataset, the data was standardized due to the very large range of predictor values. Then, the eigenvalues must be calculated to determine which linear combinations maximize the variance and are most significant. The first principal component is the normalized linear combination of the features that has the largest variance. The second principal component is the linear combination of the variables with the next highest variance out of all remaining linear combinations uncorrelated with the first principal component. This forces the second principal component to be orthogonal to the first principal component. The number of principal components is selected using eigenvalues so that they explain most of the variance in the data. The original data is then projected onto the principal components which results in a dataset with reduced dimensions. To determine the effectiveness of PCA, the Proportion of the Variance Explained (PVE) by each selected principal component is of interest. The summation of the PVE values for the principal components results in the cumulative PVE which is equal to 100%. It is ideal to select the number of principal components to be used which ensures that the majority of the variance in the data is explained. This can also be determined by visual representation through a scree plot. A scree plot simply plots PVE vs principal components (James et al., 2013).

An advantage of PCA is handling correlated variables by combining them into principal components. This leads to an uncorrelated, compact, and more significant representation of highly correlated data. PCA also produces simpler data sets by reducing their dimensions, and PCA can reduce noise by excluding insignificant variables. It is important to note that PCA causes the features to become more complex because many of them are combined into principal

components. This leads to greater difficulty in interpreting the features of the data set. PCA is also sensitive to scaling; thus, the data should be standardized.

Clustering is a method used to divide observations into distinct subgroups called clusters to discover patterns within the observations of the data. Clustering is a form of unsupervised learning. This means that the response variables are unknown, and prediction is not the goal. Rather, clustering is used to explore and learn more about the data and to find potential relationships between observations. This is referred to as Exploratory Data Analysis. Clustering aims to simplify the data by analyzing it with minimized Sum of Square Errors (SSE), and the results are typically easy to interpret. Clustering relies on a distance or dissimilarity measure to group close and similar observations. A commonly used dissimilarity measure is Euclidean Distance. Euclidean Distance is simply the length of the line segment between two points. The goal is to minimize the summation of the within-cluster variation across all clusters. Clustering can be used on observations in the data to find potential subgroups in specific attributes, or it can be used on attributes to find potential subgroups in the observations.

In K-Means Clustering, the data is divided into a pre-specified number (k) of non-overlapping clusters. Each observation belongs to exactly one cluster. The observations are randomly assigned a cluster, and the centroids of each cluster are calculated. Then, each observation is reassigned to the nearest centroid, and the centroids are recalculated. This process is repeated until the observations are no longer reassigned. One disadvantage to K-Means Clustering is that the number of clusters must be pre-specified. Conversely, in Hierarchical Clustering, the number of clusters is unknown and is determined after the observations are grouped. A tree-like diagram known as a dendrogram is used in hierarchical clustering to view the clustering for each possible number of clusters. The possible number of clusters is equal to

one less than the total number of observations. The lower the branch where two observations or groups of observations meet, the more similar those observations are to each other. The top branch includes all observations and groups them into one overarching cluster. Further down the tree, the clusters become more specific, and at the bottom of the tree, each observation is separated into its own individual cluster. This is known as Bottom-Up Clustering. Hierarchical Clustering is not effective if the potential clusters at different levels of the dendrogram are not nested.

There are some disadvantages to clustering. Regardless of the clustering method, decisions must be made and can have a strong impact on results. It is difficult to validate whether discovered clusters represent true subgroups in the data or are simply a result of noise. Clusters can be heavily influenced by outliers in the data, and they are not resistant to changes in the data. Consequently, the results should not be regarded as the absolute truth (James et al., 2013). Instead, the results should lead to the development of scientific hypotheses and further analysis.

The Rand index is a measure that is used to compare the similarities between two data clustering results. It is used to determine whether or not different clustering methods used on the data agree with each other. The Rand index is calculated by adding the number of times a pair of elements belong to the same cluster with the number of times that a pair of elements belong to a different cluster and then dividing by the total number of possible pairs of elements (Bobbitt, 2021). The Rand index values range from 0 to 1. A Rand index value of 0 indicates that the two clustering methods completely disagree on the clusters of the data set. Whereas a Rand index value of 1 indicates that the two clustering methods resulted in identical clusters and are in complete agreement on the clusters of the data set. One disadvantage of the Rand index is that it can suffer from bias if there are many clusters or if the cluster sizes vary. Thus, the Adjusted

Rand index is often used. The Adjusted Rand index is more robust and adjusts for chance when comparing the similarities between the results of two clustering methods (GeeksforGeeks, 2024). The Adjusted Rand index values range from -1 to 1. An Adjusted Rand index value of -1 indicates that the two clustering methods completely disagree with the clusters of the data set and the results are worse than chance alone. An Adjusted Rand index value of 0 indicates that the clusters were the result of random assignment and occurred completely by chance, and an Adjusted Rand index value of 1 indicates that the two clustering methods resulted in identical clusters and are in complete agreement on the clusters of the data set.

Logistic Regression is a supervised machine learning method used for binary classification. In other words, it is used when there are only two possible outcomes or classes. For classification, logistic regression predicts the probability that a given input, X , belongs to a certain class. Logistic regression is modeled by the Logistic Function:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}.$$

This function ensures that the outputs for all values of X exist within 0 and 1 which is necessary for binary classification. $X_1 \dots X_p$ are the predictor variables included in the model. β_0 represents the baseline value or the model's probability estimate without any predictor variables. $\beta_1 \dots \beta_p$ are the coefficients of the predictor variables and represent how strongly its associated predictor variable affects the output probability. A condition is specified that if the probability is greater than a certain value, the input belongs to one class, and if it is less than that value, the input belongs to the other class (James et al., 2013). In the case of this study, if $p(X) < 0.5$, then X was classified as 0 which corresponded to a team that will not qualify for the MLB postseason. If

$p(X) > 0.5$, then X was classified as 1 which corresponded to a team that will qualify for the MLB postseason. Logistic regression for classification will produce a sigmoid S-curve with a range between zero and one. One downside to logistic regression is that when the output classes are well-separated, the parameter estimates for the coefficients of the predictor variables can be highly unstable.

Logistic regression can also include higher-order terms such as squared or cubic terms by transforming the original predictors into polynomial terms. This is referred to as Polynomial Regression. Polynomial regression can allow for complex non-linear decision boundaries in the predictors, but it risks overfitting the data and not generalizing well. This can be further expanded to include Regression Splines. Regression splines allow for fitting continuous piece-wise polynomial functions. Each “piece” of the piece-wise polynomial is joined smoothly at certain points in the curve. These points are called “knots”. A type of regression spline referred to as a Natural Spline adds the constraint that the spline becomes linear at the edges or outside the boundaries. It helps prevent overfitting at the extremes of the data and can assist in stabilizing the model where less observations exist. Finally, Generalized Additive Models (GAMs) are combinations or sums of functions in which each predictor variable has its own corresponding function. These functions can be non-linear and can include regression splines. GAMs are flexible and allow for a better understanding of how each predictor variable affects the response variable. In the classification setting, GAMs may be more useful in interpretation and understanding each predictor variable rather than in accurate prediction.

Linear Discriminant Analysis (LDA) is a supervised learning method used to categorize data into two or more classes. LDA finds a linear combination of the predictor variables that best separates the classes and projects the data onto this linear combination. It attempts to maximize

between-class variation and minimize within-class variation. Essentially, it attempts to find a linear combination that results in distinct classes where the observations in each class are similar. LDA assumes that each predictor variable follows a Gaussian normal distribution meaning that each predictor variable follows a one-dimensional normal distribution, and the predictor variables can be correlated. The performance of LDA is measured with a confusion matrix. The confusion matrix compares the model's predicted outcome to the actual known outcome.

Confusion Matrix				
		<i>True (Actual) Status</i>		Total
		No	Yes	
<i>Predicted Status</i>	No	True Negative	False Negative	
	Yes	False Positive	True Positive	
Total				

True Negative refers to the number of correctly predicted observations that fall within the negative class, and True Positive refers to the number of correctly predicted observations that fall within the positive class. For this study, the negative class represents the teams that do not qualify for the MLB postseason, and the positive class represents the teams that do qualify for the MLB postseason. False Negative refers to the number of observations that the model predicted to be within the negative class, but they actually belong to the positive class. In our case, this represents when the model predicts a team did not qualify for the postseason, when the team did qualify for the postseason. Finally, False Positive refers to the number of observations that the model predicted to be within the positive class, but they belong to the negative class. In our case, this represents when the model predicts a team did qualify for the postseason, when the team did not. Error rate and accuracy are used to measure the performance of the model. Error rate refers to the number of incorrectly classified observations divided by the total number of observations, and accuracy refers to the number of correctly classified observations divided by

the total number of observations. $\text{Error Rate} = 1 - \text{Accuracy}$. Also, a “Receiver Operating Characteristics” (ROC) curve displays the types of errors for all possible thresholds (James et al., 2013). The area under the curve (AUC) ranges from 0 to 1 and gives the overall performance of a classifier over all possible thresholds. Larger AUC values indicate better performance. An AUC value of 1 represents a perfect classifier with no error. An AUC value of 0.5 represents the value received if the model was randomly guessing, and any AUC value less than 0.5 indicates that the model performed worse than simply randomly guessing the classifications.

K-Nearest Neighbors (KNN) Classification is a supervised learning method in which the classification of an observation is determined by the classification of any k number of closest observations or neighbors. The observation is then categorized into the class which contains the majority of these neighboring observations. KNN classification is computed by an algorithm beginning when the value for k (number of nearest neighbors) is chosen. For a new observation, the algorithm measures its distance from all other data points stored. It identifies the k -nearest neighboring observations and classifies the new observation into whichever class is most common among these k -nearest neighbors. To assess the performance of KNN classification, a data set is randomly divided into a training set and a test set. Typically, the training set contains 70-80% of the data, and the test set contains 20-30%. The training set is stored for use in the algorithm, and the test set are treated as “new” observations. The performance of KNN classification can be measured with a confusion matrix or through cross validation. Cross validation will be discussed later. Some benefits of KNN classification are that it is simple and easy to understand and implement. It can also react well to complex decision boundaries in the data. Some disadvantages of KNN classification are that it is sensitive to data scaling and noisy

data. It often does not generalize well, and it can struggle with high dimensionality or many predictors.

Decision Trees involves splitting a training set on the data through recursive binary splitting on the predictor variables. The resulting regions are distinct and non-overlapping. This is also known as segmenting the predictor space. Decision trees are top-down, meaning the top of the tree or the root node represents the entire training set, and more splits or decisions are made the further you move down the tree. The splits or decision points are called internal nodes, and branches are the connections between nodes. Finally, the terminal nodes or leaves are at the bottom of the decision tree. They represent when the final decision is made, and the observation is given a predicted class. The goal of recursive binary splitting is to minimize the Residual Sum of Squares (RSS). The most impactful predictor variable is selected and divided into two regions at determined cutpoint s . The cutpoint selected is the cutpoint that results in the largest reduction in RSS. Region 1 contains any observation where the value of the predictor variable is less than s . Region 2 contains any observation where the value of the predictor variable is greater than s . This process is repeated on existing nodes until each terminal node contains fewer than a specified minimum number of observations. This is a greedy, top-down approach, meaning it begins with the full training set and works its way down the predictor space making the best decision at each step without concern for future nodes (James et al., 2013). The depth of the tree is the number of splits it took to reach a terminal node from the root node. Trees with high depth may become too complex and result in overfitting the data on the training set. Tree pruning can reduce complexity and assist with generalization. Pruning is the process of selecting a subtree by removing unnecessary nodes that do not significantly improve prediction but add complexity. Optimal pruning is often determined through cross validation. The advantages of decision trees

are that they are simple and easy to interpret. They are also easily displayed graphically. The major disadvantage of decision trees is that they typically are less accurate in predicting compared to other supervised learning methods.

Random Forests and Bagging are supervised learning methods that build off decision trees. Bootstrap Aggregating, also known as Bagging, is used to improve the accuracy and stability of highly variable decision trees. Bagging involves taking many random training sets by sampling with replacement from the original training set and fitting a separate decision tree to model each training set. For each test observation, the predictions made from each of the separate decision trees are recorded. The test observation is then categorized into the most common predicted class for that observation (James et al., 2013). Bagging improves the accuracy of decision tree predictions; however, it becomes very difficult to interpret the model eliminating the advantage of decision trees. Random Forests are like Bagging; however, in Random Forests, the decision trees become decorrelated. If there is only one strong predictor variable in the data set, all the bagged trees will use this strong predictor in the first split. This will lead to all the bagged trees looking alike, and the predictions from these bagged trees will be highly correlated. Random Forests force each split in the tree to only consider a random subset of the predictors. This subset typically contains the square root of the total number of predictor variables. The results from Random Forests can be less variable and more reliable than the results from Bagging.

Cross Validation is used to evaluate the performance of machine learning models. It measures how well a model generalizes to “new” data. Cross validation can be used to detect overfitting, and it can also be used to compare the effectiveness of different models or methods in making predictions. This is typically done through test errors. The test error is calculated by

dividing the number of incorrect predictions on the test set by the total number of observations in the test set. More accurate or effective models will result in lower test errors. K-Fold Cross Validation was used to evaluate the models in this study. In k-fold cross validation, the data set is divided into K number of subsets or folds. For a given iteration, K-1 subsets are used for training, and the remaining subset is used for validation. This process is repeated K times until each subset has been used for validation exactly once. The resulting test errors are then averaged, resulting in the estimated overall test error of the model. 10 was selected as the value for k in this study. An alternative to cross validation is the Validation Set Approach. The Validation Set Approach involves separating the data set into a training set and a validation set. The model is fit on the training set. The model then attempts to predict the outcomes for the unseen validation set. The error rate of the validation set is used to estimate the test error rate (James et al., 2013).

2 Introduction & Data Collection

Machine learning is driven by vast amounts of data. As sports—particularly baseball—place greater emphasis on data collection, innovation in metrics, and statistical analysis, the application of machine learning in professional sports is becoming increasingly significant. Some examples of machine learning applications in baseball include performance prediction, scouting and player recruitment, injury prevention, team strategy, and even fan engagement. The motivation for this study stems from the first two application examples: performance prediction and team strategy. This study aims to identify factors that strongly influence a team's likelihood of reaching the

postseason. It also seeks to accurately predict teams that will qualify, assuming they meet the key performance criteria identified through the analysis.

Data was collected on the 30 MLB teams over the 1998-2024 seasons. 60 variables were considered including descriptive identifiers, total team hitting statistics for the corresponding season, and total team pitching statistics for the corresponding season. There were 810 observations in the original data set. Hitting and pitching data were exported separately from Stathead Baseball (Sports Reference, 2000). This data was then merged by the Team and Season variables and stored in Microsoft Excel. The exploration and analysis completed in this study were executed in RStudio. Thus, the “xlsx” package was installed, and the data was imported into RStudio.

This ensured that all records were correctly aligned for each team and season. The cutoff for the oldest season to be considered was 1998. This is because the 1998 season marked the beginning of the Post-Expansion Era in the MLB, making it the first season to feature the current 30-team structure. It is important to note that there have been minor changes in rules, postseason structure, and team branding and location; however, for the most part baseball has remained the same. These changes may have had small influences on statistics, though. It is also important to note that all records from the 2020 season have been removed from the data set resulting in 780 considered records. The 2020 season occurred during the height of the COVID-19 pandemic. As a result, only 60 games were played as opposed to the typical 162-game season. This had a significant impact on many of the variables listed below and does not accurately reflect patterns observed in other, full-length seasons. Due to the shortened season, the postseason was expanded to 16 teams instead of 10, and it contained more than half of the teams in the league. The identifying variables such as Rk, Season, Team, and League were excluded prior to the analysis

as they were not necessary to the modeling process or interpretation of results. However, they were included in the model predictions to allow the predicted qualifying teams to be identified. All variables included in the original data set and their descriptions are presented in the table below.

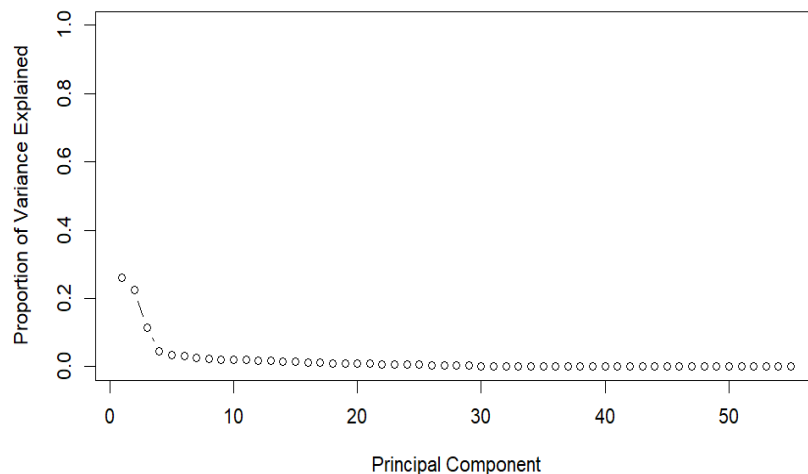
Variable Name	Description
Rk	Rank (Index)
Season	Year Season was Played
Team	Team Name
Lg	League (American/National)
GP	Games Played
W	Wins
L	Losses
WL%	Win/Loss Ratio
Bat#	Number of Batters Used
PA	Team Plate Appearances
AB	Team At Bats
R	Team Runs Scored
H	Team Hits
1B	Team Singles
2B	Team Doubles
3B	Team Triples
HR	Team Homeruns
RBI	Team Runs Batted In
SB	Team Stolen Bases
CS	# of Times Team was Caught Stealing
BB	Team Base on Balls (Walks)
SO	Team Strikeouts (Hitting)
BA	Team Batting Average
OBP	Team On Base Percentage
SLG	Team Slugging Percentage
OPS	Team On Base + Slugging Percentage

OPS+	Normalized On Base + Slugging Percentage
TB	Team Total Bases
GIDP	# of Times Team Grounded into a Double Play
HBP	# of Times Team was Hit by a Pitch
SH	Team Sacrifice Hits
SF	Team Sacrifice Flies
IBB	Team Intentional Base on Balls
LOB	# of Runners Left on Base
R/Gm	Runs Scored per Game
ERA	Team Earned Run Average
G	Games Played
CG	Team Complete Games Thrown
SHO	Team Shutouts Thrown
SV	Team Saves
IP	Innings Pitched
H.1	Team Hits Allowed
R.1	Team Runs Allowed
ER	Team Earned Runs Allowed
HR.1	Team Homeruns Allowed
BB.1	Team Base on Balls (Walks) Allowed
IBB.1	Team Intentional Base on Balls Allowed
SO.1	Team Strikeouts Thrown
HBP.1	# of Team Hit by Pitches Allowed
BK	Team Balks
WP	Team Wild Pitches
BF	# of Batters Faced
ERA+	Normalized Earned Run Average
FIP	Fielding Independent Pitching
WHIP	Walks + Hits per Inning Pitched
H9	Average # of Hits Allowed per 9 Innings Pitched
HR9	Average # of Homeruns Allowed per 9 Innings Pitched
BB/9	Average # of Walks Allowed per 9 Innings Pitched
SO/9	Average # of Strikeouts Thrown per 9 Innings Pitched
SO/BB	Team Strikeout/Walk Ratio

3 Unsupervised Learning Methods

3.1 Principal Component Analysis

The dataset contained many variables, even after some of the identification variables were removed. Thus, Principal Component Analysis (PCA) was performed. The data was scaled because some of the variables are ratios, and some of the variables are counts of certain criteria. Scaling the data ensured that the PCA would capture the actual structure of the data and not be influenced by large differences in scale. 55 Principal Components were computed using the `prcomp()` function in RStudio. The loading vectors that were used to rotate the original data to obtain the principal components were obtained. The summation of these loading vectors was equivalent to 1, so the loading vectors were normalized. A biplot was created to visualize both the principal component scores and loadings; however, the dataset was large which caused the biplot to be messy and difficult to interpret. The proportion of the variance explained (PVE) by each principal component was calculated and visualized using the scree plot below.

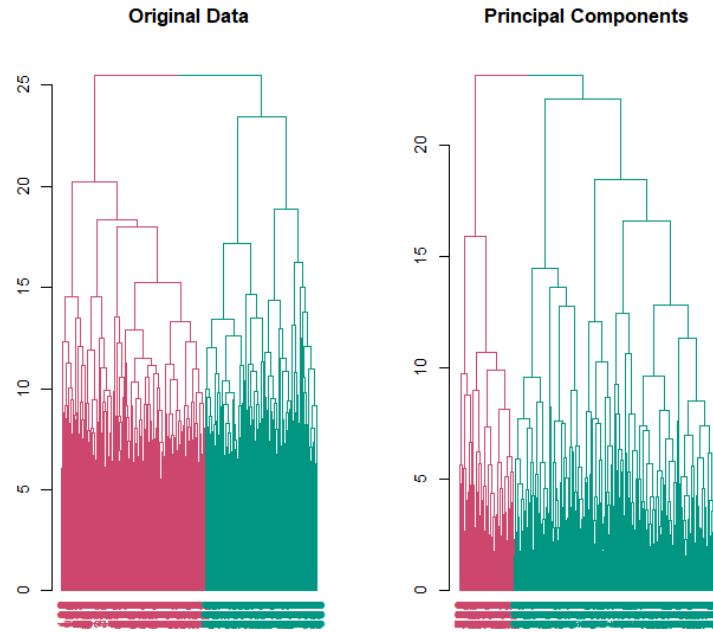


It is common to identify the “elbow” in the scree plot to determine the number of principal components that should be used. It appeared that the elbow in our scree plot was at the fourth principal component. Therefore, four principal components were used, and the total proportion of the variance explained by these four principal components was approximately 64.5%. The loadings for these four principal components were examined to determine the most significant variables, or the variables that contributed most to each principal component. The most significant hitting variables included Hits, On-Base-Percentage, On-Base Plus Slugging, Batting Average, At Bats, Strikeouts, Runs Scored per Game, Runners Left on Base, and Slugging Percentage. The most significant pitching variables included Strikeouts per 9 Innings, Earned Run Average, Normalized Earned Run Average, Walks-Hits per Inning Pitched, Walks per 9 Innings, Hits per 9 Innings, Strikeout-Walk Ratio, Homeruns per 9 Innings, and Runs Allowed. These variables were used later in supervised learning methods to attempt to create accurate models. Finally, the original scaled data was projected onto the principal scores of the four principal components, and clustering was performed using this projection.

3.2 Clustering

Hierarchical Clustering

Hierarchical Clustering with complete linkage was performed on both the original scaled data and on the principal component projection. Two distinct clusters were chosen to attempt to split the data into teams that made the postseason and teams that did not make the postseason.



Both the clustering on the original scaled data and on the principal component projection resulted in two distinct clusters which suggested underlying patterns across the teams. The dendrogram of the principal component clustering had a more defined, tree-like structure which validated the dimensionality reduction. Thus, PCA stabilized high-dimensional clustering and maintained key information. The principal component clustering also appeared to result in more distinct clusters than clustering on the original dataset. The percentage of the data contained in each cluster was calculated. Approximately 80% of the data was contained in Cluster 1 and approximately 20% of the data was contained in Cluster 2. Thus, Cluster 1 was represented by the green grouping in the dendrogram, and Cluster 2 was represented by the pink grouping in the dendrogram. The means of all the variables of the two resulting clusters from clustering on the principal components were calculated and listed below. These means can be used to compare the difference between the two groups.

	cluster	Season	L	WL	Bat.	PA	AB	R	H	X1B	X2B	
1	1	2012.525	82.86111	0.4883023	41.60458	6158.08	5505.042	714.0490	1395.412	919.9036	276.8562	
2	2	2003.839	74.08333	0.5425714	38.54762	6346.00	5610.643	845.1964	1538.000	1001.8571	310.3929	
		X3B	HR	RBI	SB	CS	BB	SO	BA	OBP	SLG	
1	28.10458	170.5474	680.1356	93.86111	35.67157	509.0539	1226.753	0.2533922	0.3209297	0.4067990		
2	30.65476	195.0952	805.9286	96.94048	38.64881	579.8690	1053.065	0.2740714	0.3453631	0.4446369		
		OPS	OPS.	TB	GIDP	HBP	SH	SF	IBB	LOB	R.Gm	ERA
1	0.7277353	95.36275	2240.119	120.9493	58.82190	42.21242	41.63725	33.20425	1114.484	4.407353	4.219395	
2	0.7899702	103.69643	2494.988	130.1190	57.75595	46.86905	50.32143	43.06548	1180.619	5.220238	4.436488	
		G	CG	SHO	SV	IP	H.1	R.1	ER	HR.1	BB.1	IBB.1
1	161.9510	3.875817	1.609477	40.54902	1442.371	1411.18	733.9706	675.8611	175.6536	521.7974	34.72876	
2	162.0119	6.523810	2.416667	41.92262	1446.687	1480.56	772.6250	712.8929	176.4940	533.4464	37.51190	
		SO.1	HBP.1	BK	WP	BF	ERA.	FIP	WHIP	H9	HR9	BB9
1	1220.358	58.83170	5.318627	55.02941	6180.685	101.1667	4.221356	1.340340	8.806699	1.095915	3.257353	
2	1076.363	57.72024	5.071429	50.29762	6263.631	103.6845	4.424821	1.392244	9.211310	1.095833	3.317262	
		SO9	SO.BB									
1	7.613235	2.385556										
2	6.694643	2.065060										

The distribution of the Win-Loss Percentage across the two clusters was also visualized through boxplots. This gave insight into whether the clustering separated the data into groups of teams that made the postseason and teams that did not. Teams that had a higher Win-Loss Percentage likely made the postseason, and the contrary is true. The blue boxplot represents the Cluster 2 or the pink cluster in the above dendrogram, and the red boxplot represents Cluster 1 or the green cluster in the above dendrogram.



K-Means Clustering

The Within-Cluster Sum of Squares (WSS) was calculated for one to ten clusters on the principal component projection. WSS measures how compact the clusters are by determining how close each data point in the clusters is to their own cluster's centroid. The goal was to minimize WSS while maintaining simplicity and interpretability. Thus, $k=2$ clusters were selected. The results of the k-means clustering on the first and second principal components are displayed below.



The k-means clustering with $k=2$ clusters was performed with one random start and with 20 random starts, and the total WSS between the two trials was compared. With 20 random starts, k-means was computed 20 times, each with a different random starting centroid, and the best clustering result, or the result that contained the lowest WSS, was returned. The k-means clustering trial with one random start resulted in a WSS value of 20168.8, and the k-means clustering trial with 20 random starts resulted in a WSS value of 20165.59. The trial with 20

random starts resulted in a lower WSS value, so this trial was considered slightly more reliable. There was an exceptionally small difference in WSS, so the cluster structure on the principal component projection is clear and stable. It also suggested that the clustering was robust to initialization randomness and that k-means converged quickly and could reliably detect groupings within the data.

3.3 Method Comparison

The “cluster”, “mclust”, and “fossil” packages were installed and loaded into the R file. The rand index score was calculated to compare the hierarchical clustering results and the k-means clustering results. The rand index score was found to be 0.5666206. This means that the hierarchical clusters and the k-means clusters on the clustering of approximately 56.7% of the observations in the dataset. This is slightly better than random and shows that the two methods agree on some structure; however, the two methods still result in notably different groupings. The adjusted rand index was then computed to consider the agreement between the two methods due to chance. The adjusted rand index score was found to be approximately 0.1329. This confirmed that although the two clustering methods resulted in more similar results than through randomness, they produced significantly different clusters and likely captured different underlying structures in the data. The difference between the results of the two clustering methods was likely due to the differences between how the clusters were formed. Hierarchical clustering with complete linkage forms clusters based on the maximum distance between points from different clusters while k-means clustering forms clusters by assigning observations to the nearest randomly selected centroid which is updated iteratively. Due to the disagreement

between the clustering methods, it was decided that the results would be used for model interpretability and for identifying the most important features in the dataset to reduce dimensionality.

4 Supervised Learning Methods

A categorical variable denoting whether each team made the postseason was created, filled with values, and added to the dataset. This variable was labeled “Playoffs”, and this variable was used as the response variable to attempt to categorize teams that did or did not make the postseason. The only values of this Playoffs variable were “Yes” which corresponded to a team that did qualify for the postseason and “No” which corresponded to a team that did not qualify for the postseason. The Playoffs variable was treated as a factor to ensure that it could be used in the different supervised learning models. Many different models were fit on the data to find one that could accurately classify and predict the teams in an MLB postseason. The seed was set using “set.seed(123)” for reproducibility. The new dataset was scaled and divided into a training set and a test set. The training set contained 80% of the 810 observations, and the test set contained the remaining 20% of the observations.

4.1 Logistic Regression

First, a full logistic regression model was fit onto the training set. The full model contained 57 predictor variables after non-necessary descriptive variables were removed from the dataset. Only 9 of these variables were found to be significant. These variables included: RBI, OPS+,

IBB, HR.1, SO.1, BF, FIP, BB/9, and SO/9. The prediction accuracy of the full model was then evaluated by using the full model to classify the observations in the test set. If the probability of an observation containing a Playoffs value of “Yes” was greater than 0.5, then the observation was classified as qualifying for the postseason. If the probability was less than or equal to 0.5, then the observation was received “No” for the Playoffs response variable and classified as not qualifying for the postseason. The accuracy of the full model was then calculated through a confusion matrix and was found to be 0.87654432. Thus, the test error was 0.12345568. This means that the full model correctly classified approximately 87.65% of the observations in the test set and incorrectly classified approximately 12.35%. This process was repeated for two different reduced models. The first reduced model contained only 18 predictor variables found to be important through PCA and clustering. Thus, the first reduced model was trained using SO, AB, OBP, H, BA, OPS, R/Gm, LOB, SLG, SO/9, ERA, ERA+, WHIP, BB/9, H/9, R.1, HR/9, and SO/BB. Only OBP, H, BA, OPS, R/Gm, SLG, ERA, SO/9, and R.1 were found to be statistically significant. The accuracy of the model was also evaluated on the test set through a confusion matrix and was found to be 0.8888889, and the test error was 0.1111111. Finally, the second reduced model contained only the predictor variables found to be statistically significant in the first reduced model. The model was fit on the training set, and only OPS, R/Gm, ERA, and SO/BB were significant. The accuracy of this model was found to be 0.9012346, and the test error was found to be 0.0987654.

4.2 KNN Classification

KNN Classification was performed on the dataset several times with different values for k . The values for k included 1, 2, 3, 4, 5, 7, and 10. All actual classifications of the Playoffs response variable from both the training set and test set were stored in two newly created variables named “train_labels” and “test_labels”. The train_labels variable was used to train the model and assist it in classifying the test data. The test_labels variable was used to evaluate each model. Each KNN Classification model was trained using the training set and evaluated by the success of its classifications of the test set similar to the evaluation of the logistic regression models. When $k=1$ or 2, KNN Classification resulted in an accuracy of 0.8209877 and a test error of 0.1790123. When $k=3$, it resulted in an accuracy of 0.845679 and a test error of 0.154321. When $k=4$, 5, or 7, it resulted in an accuracy of 0.8641975 and a test error of 0.1358025. Finally, when $k=10$, KNN Classification resulted in an accuracy of 0.8703704 and a test error of 0.1296296. The accuracy and test error values for the different KNN Classification models would vary for different iterations; however, the accuracy values would likely remain close to 80-85%, and the test error values would likely remain close to 15-20%. The values do not vary for this study because the seed was set.

4.3 Linear Discriminant Analysis

The “MASS” package was installed and loaded into RStudio. Three different LDA models were considered. These models contained the same predictor variables as the three logistic regression models considered. Thus, the first LDA model was a full model and contained all the predictor variables. The second LDA model was a reduced model that contained the same 18

predictor variables as the first reduced logistic regression model. The third LDA model was a further reduced model that contained the same predictor variables as the second or most reduced logistic regression model. Similarly to the logistic regression and KNN classification models, the three LDA models were trained on the training set and evaluated by predicting the classifications of the Playoffs variable of the observations in the test set. The accuracy and test error values were also computed through confusion matrices. The full LDA model resulted in an accuracy value of 0.962963 and a test error value of 0.037037. The first reduced LDA model resulted in an accuracy of 0.9135802 and a test error of 0.0864198. Finally, the second or furthest reduced LDA model resulted in an accuracy of 0.9012346 and a test error of 0.0987654. The LDA models were also evaluated using 10-fold Cross Validation. 10-fold Cross Validation produced similar accuracy and test error values. This suggested that the LDA models generalized well to unseen data.

4.4 Non-Linear Functions

Polynomial Regression

The Playoffs variable was plotted against the 18 important variables identified through PCA to determine if non-linear relationships existed between the response variable and significant predictor variables. Therefore, the Playoffs variable was plotted against AB, OBP, H, BA, OPS, R/Gm, LOB, SLG, SO/9, ERA, ERA+, WHIP, BB/9, H/9, R.1, HR/9, and SO/BB. It was difficult to determine if there were non-linear relationships through the plots because the response variable is categorical and not numerically continuous. All 18 predictor variables were considered for non-linear relationships. Linear, Quadratic, Cubic, 4th-Degree, and 5th-Degree

Polynomial models were fit on each individual predictor variable. A Chi-Squared test was performed to compare the five polynomial models for each predictor variable. The p-value for the cubic polynomial for the SO/BB variable was 0.04331 and showed evidence of a potential cubic relationship with the Playoffs response variable. All other polynomial functions for the remainder of the predictor variables showed no evidence of non-linear relationships. The cubic polynomial model of the SO/BB variable was evaluated through 10-fold cross validation. It resulted in an accuracy of 0.8198794 and a test error of 0.1801206.

Splines & Natural Splines

The “splines” package was installed and loaded into RStudio. The SO/BB variable was the only predictor variable that showed evidence of a non-linear relationship with the Playoffs response variable. Therefore, the SO/BB variable was the only variable used to form the spline and natural spline models. First, a cubic B-spline model with 6 degrees of freedom was fit on the training set using the SO/BB as the only predictor variable. 10-fold cross validation was then used to evaluate the model. The cubic spline model resulted in an accuracy of 0.8204521 and a test error of 0.1795479. Then, a natural cubic spline model with 4 degrees of freedom was fit on the training set using the So/BB as the only predictor variable. The test error and accuracy of the natural cubic spline model was also calculated using 10-fold cross validation. The natural cubic spline model resulted in an accuracy of 0.820517 and a test error of 0.179483.

Generalized Additive Models

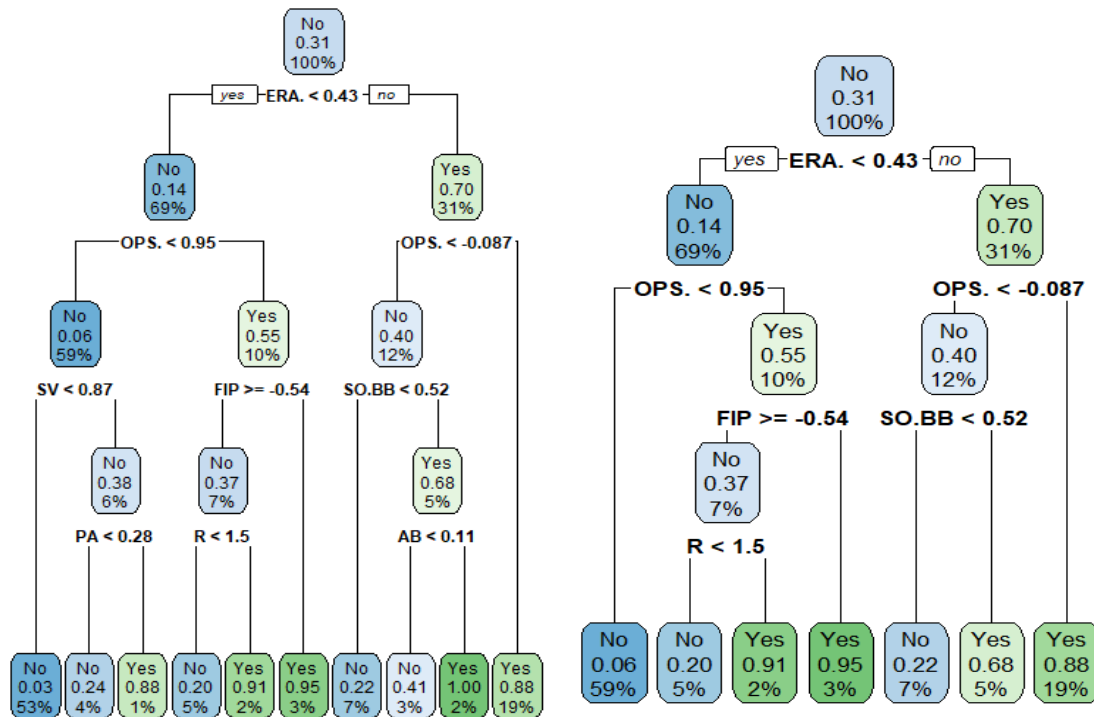
Similarly to the splines, a generalized additive model was fit on the training data using only the SO/BB predictor variable. This was problematic because there was only evidence of one potential non-linear relationship in the dataset. Generalized additive models specialize in adding

many non-linear models on an individual predictor variable such as splines or natural splines into a singular model. For this study, a generalized additive model fit a natural spline model of the SO/BB predictor variable onto the training set. This model was also evaluated through 10-fold cross validation and resulted in an accuracy of 0.8206803 and a test error of 0.1793197. There seemed to be a lack of complex non-linear relationships in this data; however, it was extremely difficult and tedious to determine if non-linear relationships existed in the data because this study is focused on a classification problem and is concerned with a categorical variable. This likely led to the polynomial, spline, natural spline, and generalized additive models to perform worse than previous models.

4.5 Decision Trees, Random Forests, & Bagging

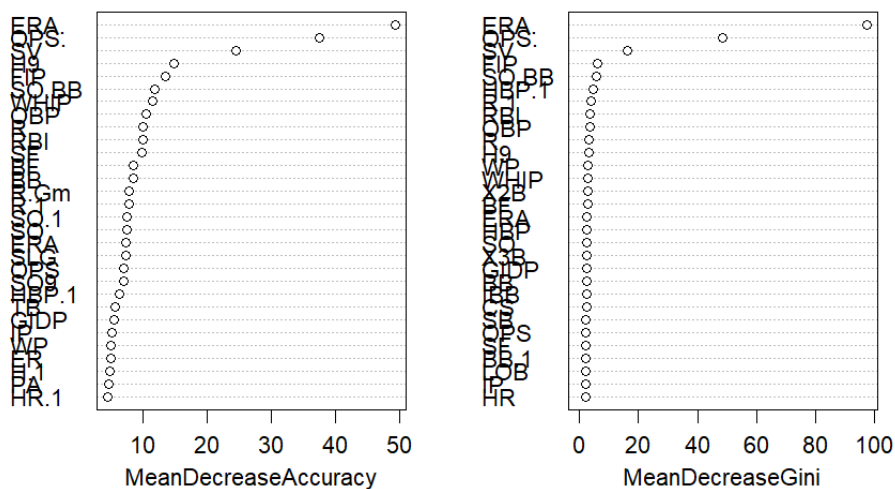
A full decision tree model was fit on the training set. The full decision tree used only the Win-Loss % variable to predict which teams would qualify for the postseason. This is entirely valid and logical as the teams with the highest winning percentages qualify for the postseason, and those with lower winning percentages do not. However, this was not the goal. This study aimed to identify factors that were correlated with or typically led to higher winning percentages and thus led to postseason qualification. Therefore, the Win-Loss %, Wins, and Losses columns were removed from the dataset, and a new decision tree model was fit on the training set. The full tree was pruned to simplify the structure and prevent overfitting. The pruned tree kept only the most important decision branches that led to the smallest test error. Both the full and the pruned tree are displayed below. The full tree is on the left, and the pruned tree is on the right. The percentages in the decision tree nodes represent the percentage of observations that fall

within that specific node or decision. Both the full tree and the pruned tree resulted in an accuracy of 0.68981 and a test error of 0.31019.



To improve the prediction accuracy of the decision trees, Bagging and Random Forest was performed. The “randomForest” package was installed and loaded into RStudio. Two new variables were created to store the training set and the test set used for Bagging and Random Forest. These training and test sets were identical to the original training and test sets; however, the Win-Loss %, Wins, and Losses predictor variables were removed. Bagging was performed on the training set using a full model, and variable importance plots were constructed. The most important variables are listed at the top of these plots and result in the higher mean decrease in accuracy values based on how much worse the model performs when the variable is moved or changed and higher mean decrease in node impurity based on how much each variable reduces the impurity. The variable importance plots are displayed below.

Bagging



These variable importance plots suggest that ERA+, OPS+, SV, FIP, and SO/BB are the most important variables. ERA+ refers to the normalized Earned Run Average. It adjusts the ERA statistics for the league average and to include the impact that each team's ballpark had on their pitchers' performance by including a park factor in the calculation. Similarly, OPS+ refers to the normalized On-Base % Plus Slugging %. OPS+ also adjusts OPS for the league average and to include the impact that each team's ballpark had on their hitters' performance by including a park factor in the calculation. The park factor is typically calculated by dividing the ratio of runs scored at home plus runs allowed at home over the number of home games played by the ratio of runs scored away plus runs allowed away over the number of away games played. A park factor of 1 typically means that the park is neutral towards offense and defense. A park factor greater than 1 means that the park is "hitter-friendly" or typically results in a higher offensive output. A park factor less than 1 means that the park is "pitcher-friendly" or typically results in a lesser offensive output (Sports Reference, 2000). The equations for how the park factor, ERA+, and OPS+ are calculated are included below.

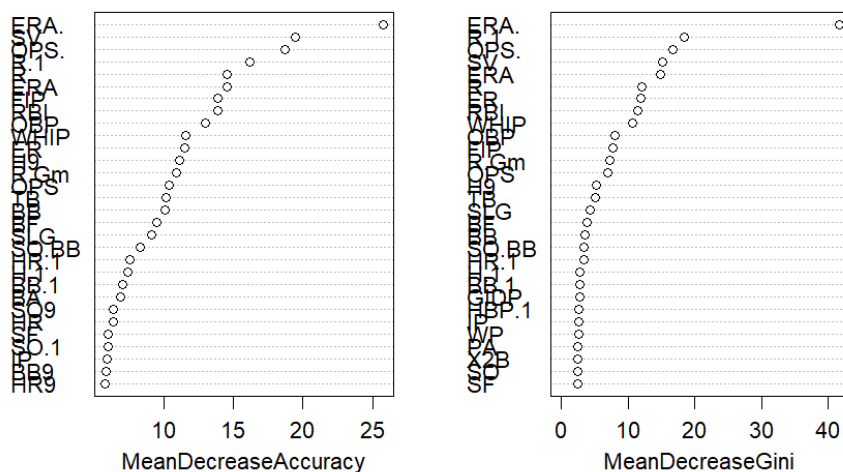
$$\text{Park Factor (PF)} = \frac{(\text{Runs scored at home} + \text{Runs allowed at home})/\text{Home games}}{(\text{Runs scored away} + \text{Runs allowed away})/\text{Away games}}$$

$$\text{ERA+} = 100 \times \left(\frac{\text{League ERA}}{\text{Pitcher's ERA}} \right) \times \left(\frac{1}{\text{Park Factor}} \right)$$

$$\text{OPS+} = 100 \times \left(\frac{\text{Player's OBP}}{\text{League OBP}} + \frac{\text{Player's SLG}}{\text{League SLG}} - 1 \right) \times \frac{1}{\text{Park Factor}}$$

The accuracy and test error were computed for Bagging. Bagging resulted in an accuracy of 0.845679 and a test error of 0.154321. Random Forest was then performed on the training set using a full model, and variable importance plots were constructed. These variable importance plots are displayed below.

RandomForest



According to the variable importance plots for Random Forest, ERA+, SV, OPS+, R.1, and RBI are the most important variables. Both the Bagging and Random Forest methods agree that ERA+, SV, and OPS+ are among the most important variables in predicting postseason

qualification. Both methods also agree that ERA+ is the most important predictor variable. The accuracy and test error for Random Forest was then computed. Random Forest resulted in an accuracy of 0.882716 and a test error of 0.117284.

5 Model Evaluation & Selection

Throughout this study many different supervised learning models were developed and fit on the data to accurately predict which MLB teams qualify for the postseason and which teams do not. Some of the models included Logistic Regression, KNN Classification, Linear Discriminant Analysis, Non-Linear Models, and Decision Trees. Each of the models were evaluated through the Validation Set Approach or through 10-fold Cross Validation. The better performing models minimized the test error and generalized well to unseen data. A table of each model and the resulting test error is displayed below.

<i>Model</i>	<i>Test Error</i>
Logistic Regression Full	0.123456
Logistic Regression Reduced	0.111111
Logistic Regression Furthest Reduced	0.098765
KNN Classification; k = 1 or 2	0.179012
KNN Classification; k = 3	0.154321
KNN Classification; k = 4, 5, or 7	0.135803
KNN Classification; k = 10	0.129630
LDA Full	0.037037
LDA Reduced	0.086420
LDA Furthest Reduced	0.098765
Cubic Polynomial	0.180121

Cubic B Spline	0.179548
Natural Cubic Spline	0.179483
Generalized Additive Model	0.179320
Full Decision Tree	0.310190
Bagging	0.154321
Random Forest	0.117284

The KNN Classification and Non-Linear models performed poorly compared to many of the rest of the models. The full Decision Tree model provided valuable insights into how some of the predictor variables affected the response variable and was easily interpreted; however, it did not result in accurate predictions and did not generalize well to unseen data. The furthest reduced Logistic Regression performed better than the other Logistic Regression models, and the full LDA model performed better than the other LDA models. Finally, Random Forest performed better than Bagging due to the ERA+ predictor variable being considered as far more important than the other predictor variables which affected how well the Bagging model generalized to unseen data. Thus, the model selection was narrowed down to the furthest reduced Logistic Regression model, full LDA model, and the Random Forest model. While the Logistic Regression and LDA models performed better and are easier to interpret than the Random Forest model, there was concern with the multicollinearity of the predictor variables. Many of the predictor variables are likely highly correlated, and Logistic Regression and LDA are sensitive to multicollinearity. Thus, multicollinearity reduction measures and further analysis would need to be conducted in order to completely trust the results of these models. Random Forest is robust to multicollinearity and outliers, accurately captures complex interactions, and works well with categorical variables. The Random Forest model also provided variable importance plots which allowed for a better understanding of predictor variables that had a significant impact on the

classification of the Playoffs response variable. Finally, Random Forest handles high dimensionality and generalizes well. Therefore, the Random Forest model was selected as the best model and was used to predict teams that would make the postseason for the 2025 season. The downside to Random Forest is that the model itself is difficult to interpret; however, it provides insight into important features.

6 Prediction & Discussion

Data was collected on the 2025 MLB season on April 12th. This data was thrown into the Random Forest model to attempt to predict the teams that will qualify for the 2025 postseason. It is important to note that as of April 12th, most of the teams had only played around 15 games. This is approximately one tenth of a typical MLB season; therefore, the predictions are likely not very accurate this early into the season. The Random Forests model predicted the 10 most likely teams to make the postseason are the Oakland Athletics, Houston Astros, Detroit Tigers, Boston Red Sox, New York Yankees, Los Angeles Dodgers, San Diego Padres, Chicago Cubs, Atlanta Braves, and New York Mets. These results were adjusted to align with the MLB postseason structure. Thus, five teams were required to come from both the National and the American Leagues. Also, at least one team had to come from each division. For the most part, the predictions seem logical. As of April 12th, many of these teams are division leaders or not far from it. The Random Forest model had a high accuracy on the test data, so it would be interesting to run the model on the prediction data later in the season and at the end of the season to test how well it predicted the 2025 postseason picture. Through PCA, the significance of variables in logistic regression, and variable importance plots of Random Forest, ERA+, Runs

Allowed, OPS+, and Runs Scored have a significant impact on whether a team would qualify for the postseason according to this study.

There are some concerns with the study. The dataset seems to have many complex underlying structures, and it likely suffers from high multicollinearity. There are also many predictor variables. For future analysis, measures should be taken to reduce the multicollinearity and high dimensionality of the data. The distribution of the predictor variables should also be examined, and assumptions should be tested. Many different models in this study resulted in high prediction accuracy, and it would be interesting to observe how well they predict the postseason for future seasons. It would be interesting to conduct further research into the ratio of runs scored and runs allowed to determine if there were a specific ratio that would greatly increase the odds of qualifying for the postseason and that teams should strive to achieve. Finally, it would be fascinating to extend this analysis to evaluate individual players across the league.

As mentioned earlier, this study found ERA+, Runs Allowed, OPS+, and Runs Scored to be significant contributors to whether an MLB team qualifies for the postseason. This shows that teams must be strong both offensively and defensively. However, considering ERA+ was continuously selected as the variable with the highest importance, strong pitching and defense may have more influence than a strong offense. This could give front offices and managers strategic insight into how to build and effectively manage their team. This insight could influence draft picks, free agent signings, trades, and player development. It could also play an important role in scouting. Potentially the most important effect would be on team strategy. With these insights, owners and general managers could strategically invest money into players that positively impact these metrics.

7 Conclusion

This study aimed to identify factors that significantly contribute to whether an MLB team qualifies for the postseason through machine learning. Through unsupervised learning methods such as clustering, patterns among team performance metrics were identified. Different supervised learning methods were utilized to form many models and evaluate their predictive accuracy. Across the many different methods used to identify important variables, the results consistently emphasized ERA+, Runs Allowed, OPS+, and Runs Scored as significant contributors to postseason qualification. This stresses the importance of teams balancing offensive and defensive strength and not focusing on one area.

Due to its prediction accuracy and ability to handle multicollinearity, the Random Forest model was selected to predict the outcomes of the 2025 MLB season based on early-season performance data. While most of the models performed well on the test data, limitations such as multicollinearity and the exclusion of external variables should be acknowledged. Future research could expand further and incorporate or evaluate individual player metrics. This study demonstrates the power of machine learning and data-driven approaches to solving complex problems. It also highlights the value that data analysis offers for sports in evaluating team performance and predicting team success.

8 References

- GeeksforGeeks. “Rand-Index in Machine Learning.” *GeeksforGeeks*, 22 Apr. 2024, www.geeksforgeeks.org/rand-index-in-machine-learning/.
- James, Gareth, et al. *An Introduction to Statistical Learning with Applications in R*. Springer New York, 2013.
- MLB. “MLB Postseason 2024: Playoff Bracket and World Series Schedule.” *MLB.com*, www.mlb.com/postseason. Accessed 1 May 2025.
- R. “Randomforest: Breiman and Cutlers Random Forests for Classification and Regression.” *The Comprehensive R Archive Network*, Comprehensive R Archive Network (CRAN), 22 Sept. 2024, cran.r-project.org/package=randomForest.
- Sports Reference. “Team Batting Season Stats Finder - Baseball.” *Stathead Baseball*, 2000, www.stathead.com/baseball/team-batting-season-finder.cgi?request=1&year_min=2005&year_max=2024&team_success=is_playoff_team.
- Zach Bobbitt. “What Is the Rand Index? (Definition & Examples).” *Statology*, 16 Nov. 2021, www.statology.org/rand-index/.